# Proyect_One_Instance_01

# Enunciado

Una pequeña empresa de hostelería le gustaría alojar su página web estática en un VPC, debe contar con un firewall básico (Solo puertos 22, 80 y 443 abiertos).

Objetivos del aprendizaje: Creación de instancias, acceso mediante SSH, firewalls a nivel de proveedor y configuración básica de un servidor web Nginx.

# Herramientas

- Terraform: Para el despliegue de infraestructura como código (IaC).
- Github: Para el control de versionado.
- CentOS: Como máquina virtual usada para la ejecución de código terraform, para el despliegue de la infraestructura.

# Instalación de la herramienta de control de versionado

**Instalación del CLI para interactuar mediante línea de comandos con GitHub**

```
sudo dnf install -y 'dnf-command(config-manager)'
sudo dnf config-manager --add-repo https://cli.github.com/packages/rpm/gh-cli.repo
sudo dnf install -y gh
```

**Autorización para el acceso a Github**

```
gh auth login
```

**Creación ficheros inicio**

Creamos la carpeta principal de proyectos.

```
cd hetzner-architecture-cloud
```

Creamos la carpeta donde creamos los ficheros de configuración, para el despliegue automático de la infraestructura.

```
cd nivel1-single-instance
```

Establecemos el fichero de configuración github

```
cat > .gitignore << 'EOF'
*.tfstate
*.tfstate.backup
.terraform/
.terraform.lock.hcl
terraform.tfvars
crash.log
EOF
```

Iniciamos el repositorio github y realizamos nuestro primer comit.

```
git init
git add .
git status
git commit -m "Nivel 1: instancia única Nginx + Terraform en Hetzner"
```

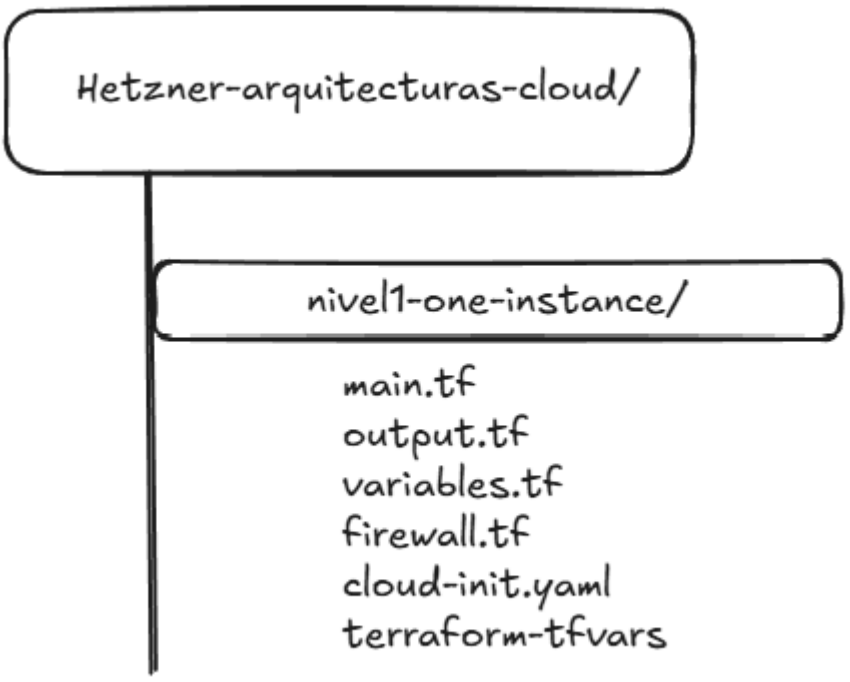
Creamos el repositorio github

```
gh repo create hetzner-arquitecturas-cloud --public --source=. --
remote=origin --push
```

Los parámetros en cuestión son los siguientes:

- `--public` (o `--private` si prefieres que no sea visible mientras aprendes)
- `--source=.` usa el directorio actual como contenido del repo
- `--remote=origin` configura el remoto automáticamente
- `--push` sube el commit inicial en el mismo paso

**Estructura del directorio**




→ main.tf: Almacena la configuración global de las instancias, es el fichero donde se declara el provider de Hetzner y los recursos principales. Así como la imagen, tipo, ubicación y configuración de la red.
→ variables.tf: Define las variables usadas en el proyecto
→ terraform.tfvars: Variables de entorno, únicamente visibles en la máquina donde se arranca el proyecto, nunca va a git.
→ firewall.tf: Definición de las reglas del firewall a nivel de proveedor. En caso de alojar una máquina virtual con firewall propio se deberá desactivarlo.
→ cloud-init.yaml: Script de aprovisionamiento automático de infraestructura, el cual se ejecuta una única vez.
→ outputs.tf: Define qué datos de la infraestructura creada quieres que Terraform te muestre al terminar (la IP pública asignada, la IPv6 etc...). Te permite consultarlos después con Terraform output.

# Ejecución del proyecto

## Instalación de la herramienta terraform

```
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo
https://rpm.releases.hashicorp.com/RHEL/hashicorp.repo
sudo dnf install -y terraform
```

## Instalación de la herramienta CLI hcloud (útil para depurar sin salir a la consola web)

```
wget https://github.com/hetznercloud/cli/releases/latest/download/hcloud-linux-amd64.tar.gz
tar -xzf hcloud-linux-amd64.tar.gz
sudo mv hcloud /usr/local/bin/
```

### Configuración de las SSH key dedicada al proyecto (No reutilizar key personal)

Se genera un par de claves (Pública y Privada)

```
ssh-keygen -t ed25519 -f ~/.ssh/hetzner_lab -C "hetzner-lab"
```

Para evitar acceder por ssh a la instancia, continuamente se introducen los siguientes comandos. Con el fin de desbloquear la clave privada en memoria.

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/hetzner_lab
```

Como veremos posteriormente, tras ejecutar el comando terraform apply para el despliegue de la infraestructura. Lo que va a ocurrir con este par de claves es lo siguiente:

1. Terraform sube tu **clave pública (hetzner_lab.pub)** a la cuenta de Hetzner.
2. Cuando Hetzner crea la instancia, **inyecta esa clave pública** en **~/.ssh/authorized_keys** del usuario root, automáticamente, antes de que la instancia arranque por primera vez.
3. La **clave privada** (hetzner_lab, sin la extensión **.pub)** se queda solo en la máquina CentOS. Tras ejecutar

```SHELL
ssh -i ~/.ssh/hetzner_lab root@<ip-del-servidor>
```

Antes de pasar a la configuración del proyecto, se debe de tener en cuenta que previamente se debe de haber creado un proyecto de trabajo en hetzner y también haber configurado la API Key correspondiente. Esta API Key se creará dirigiendonos al apartado **Security** de nuestro proyecto.

# Archivos de configuración

## cloud-init.yaml

```
#cloud-config
package_update: true
packages:
  - nginx

write_files:
  - path: /var/www/html/index.html   # en Ubuntu, la ruta por defecto de
Nginx es /var/www/html, no /usr/share/nginx/html
    content: |
      <html>
        <head><title>Nivel 1 - Terraform + Hetzner</title></head>
        <body><h1>Funciona. Nivel 1 desplegado con IaC.</h1></body>
      </html>

runcmd:
  - systemctl enable nginx
  - systemctl start nginx
```

En este fichero de configuración basicamente definiremos los paquetes con los que va a arrancar nuestra instancia en cloud, le diremos que en la ruta **/var/www/html/**, se va a escribir el contenido html en el fichero **index.html**. Le definiremos también, con qué servicios va a arrancar.

**Tip**

Nginx es un software de servidor web de código abierto, es usado para cuando se requiere un rendimiento alto. Es decir, Ngnix permite reducir el INP (Interaction to next Paint) deforma indirecta al mejorar la velocidad de entrega de recursos, reducir latencia de red, etc....

Curiosamente también puede funcionar como un servidor web, proxy inverso y balanceador de carga.

## firewall.tf

```
resource "hcloud_firewall" "basic" {
  name = "nivel1-basic-fw"

  rule {
    direction  = "in"
    protocol   = "tcp"
    port       = "22"
    source_ips = ["0.0.0.0/0", "::/0"]
  }

  rule {
    direction = "in"
    protocol = "icmp"
    source_ips= ["0.0.0.0/0", "::/0"]
  }

  rule {
    direction  = "in"
    protocol   = "tcp"
    port       = "80"
    source_ips = ["0.0.0.0/0", "::/0"]
  }

  rule {
    direction  = "in"
    protocol   = "tcp"
    port       = "443"
    source_ips = ["0.0.0.0/0", "::/0"]
  }
}
```

En este fichero definiremos las reglas con las que filtraremos el tráfico de entrada y salida, de nuestra instancia virtual. Hetzner proporciona la capacidad de crear firewall virtuales. Los cuales obviamente, también podremos definirlos en nuestro fichero de configuración Terraform para desplegarlos de forma automática.

## main.tf

```
terraform {
      required_providers {
        hcloud = {
          source  = "hetznercloud/hcloud"
          version = "~> 1.45"
        }
      }
    }

    provider "hcloud" {
      token = var.hcloud_token
    }

    resource "hcloud_ssh_key" "lab_key" {
      name       = "lab-key"
      public_key = file(var.ssh_public_key_path)
    }

    resource "hcloud_server" "web" {
      name        = var.server_name
      server_type = var.server_type
      image       = "ubuntu-26.04"
      location    = var.location
      ssh_keys    = [hcloud_ssh_key.lab_key.id]

      user_data = file("${path.module}/cloud-init.yaml")

      public_net {
        ipv4_enabled = true
        ipv6_enabled = true
      }

      firewall_ids = [hcloud_firewall.basic.id]
    }
}
```

En este fichero se almacenará la configuración global de las instancias que se van a desplegar. Definiremos el provider, en este caso hetzner. Es necesario puntualizar aquí, que nuestro provider puede ser nosotros mismos. Imaginemos la situación en la que tenemos una empresa con un rack, compuesto de varias servidores que mantienen nuestro negocio (Infraestructura on-promise). Terraform no se limita únicamente a desplegar infraestructura de forma automática en entornos Cloud. Sino también en local. Esto lo cubriremos en proyectos posteriores.

Por ahora centremonos en el hecho, de que main.tf es el core de nuestro proyecto terraform, además definir el provider. También se definirá el token que hemos creado,

para comunicarnos con la API de hetzner y por supuesto, se definirá el nombre del servidor, qué tipo de servidor es, localización etc...

Por defecto, al crear una instancia en hetzner, a esta se la dotará de una IP pública, la cual cuesta aproximidamente 0.50 € al mes.

## output.tf

Define qué datos de la infraestructura creada quieres que Terraform te muestre al terminar (la IP pública asignada, la IPv6 etc...). Te permite consultarlos después con Terraform output.

```
output "server_ip" {
  value = hcloud_server.web.ipv4_address
}

output "server_ipv6" {
  value = hcloud_server.web.ipv6_address
}
```

Un ejemplo de uso del fichero output.tf, podría ser el siguiente

```
ssh -i ~/.ssh/hetzner_lab root@$(terraform output -raw server_ip)
```

## variables.tf

```
variable "hcloud_token"{
    description= "Token API de Hetzner Cloud"
    type = string
    sensitive = true
}

variable "ssh_public_key_path" {
    default = "~/.ssh/hetzner_lab.pub"
}

variable "server_name" {
    default = "nivel1-one-instance"
}

variable "server_type" {
    default = "cx23"
}

variable "location" {

    default = "hel1"

}
```

Este archivo define las variables usadas en el proyecto. En nuestro caso, tendrá las siguientes variables:

- **hcloud_token** → Con sensitive=true, esto evita que la API key salga en pantalla a la hora del despliegue.
- **ssh_public_key_path** → Es la clave pública que se va a inyectar en la cuenta de hetzner creada.
- **server_name** → Es el nombre del servidor
- **server_type** → Tipo del servidor, hetzner ofrece servidores muy económicos, estos son económicos debido a que usan hardware antiguo. Lo que les permite abaratar costes.
- **location** → Localización del servidor.

⚠ **Warning**

Es necesario tener mucho cuidado, y no confundir conceptos. La API key que te proporciona el proveedor hetzner, no debe ir en el campo description de la variable **hcloud_token** debe de estar configurada como una variable de entorno en un

fichero con extensión tfvars. Esto evita que github, exponga la API key, bajo ningún concepto se debe de exponer la API key al público.

## Paso final: Despliegue de la infraestructura

Para llevar a cabo el despliegue de la infraestructura, lo primero que haremos será preparar el directorio con el siguiente comando

```
terraform init
```

Tras iniciar el entorno terraform en el directorio del proyecto, es recomendable revisar toda la configuración aplicada a la infraestructura que se va a desplegar. Esto evita que nos llevemos sustos en la factura.

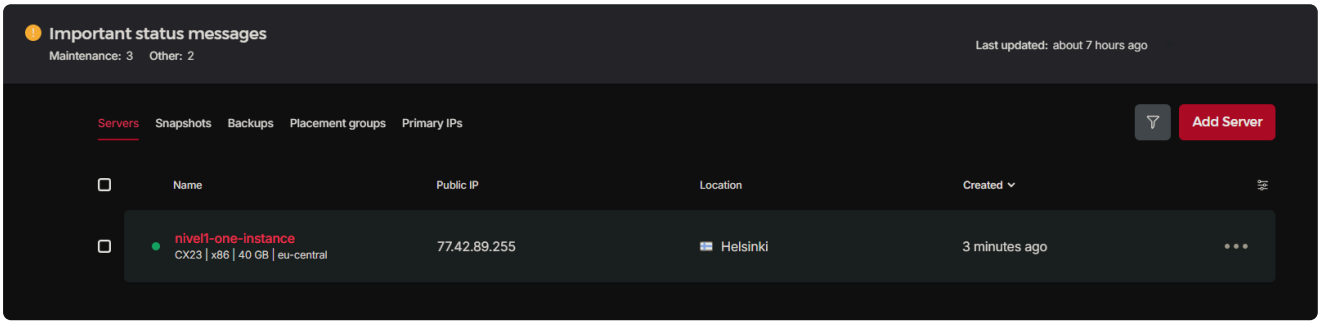
```
terraform plan
```

Si todo esta correcto, se ejecutará apply, para desplegar la infraestructura bajo la configuración aplicada.

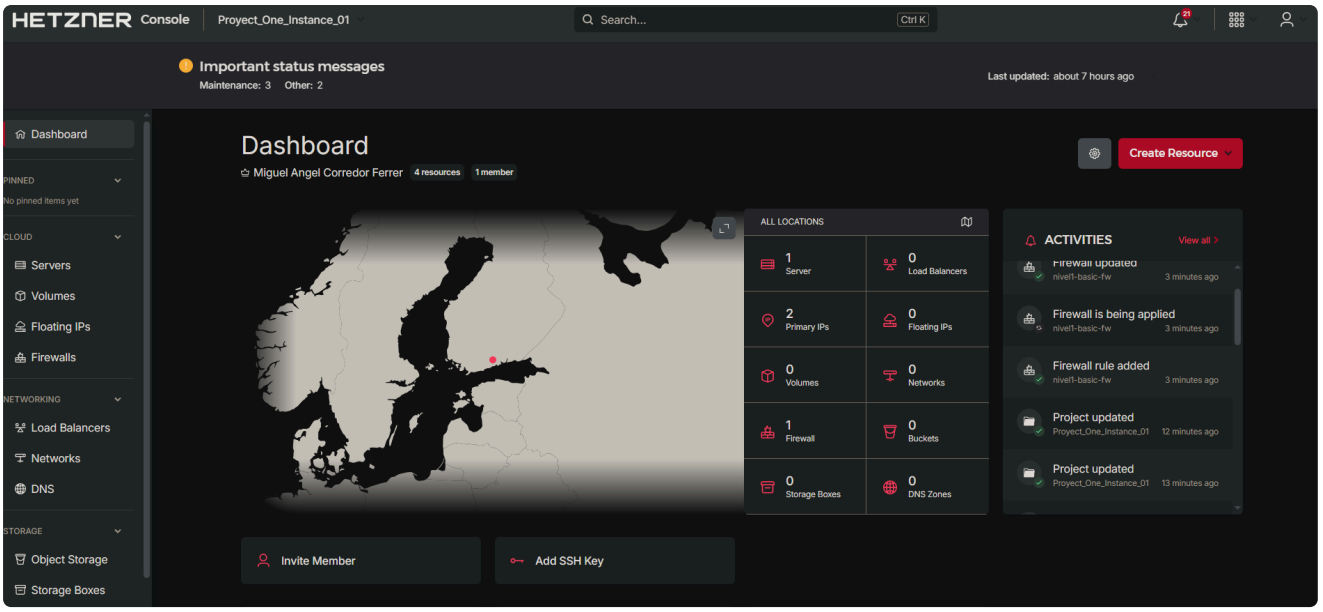
```
terraform apply
```

## Resultados del proyecto

Como vemos, los resultados son los siguientes. Se ha creado una instancia virtual ubuntu, con arquitectura x86, 4 GB de RAM y 40 GB de almacenamiento. La IP pública es 77.42.89.255 y su localización es Helsinki (Finlandia).




Este es el dashboard principal, donde se controlará los parámetros principales de nuestro proyecto. En futuros proyectos, se profundizará en cada uno de los apartados de la dashboard.




A través de la CLI de hcloud, se puede hacer un listado de las instancias actualmente en ejecución. Es necesario haber exportado la API Key en nuestra sesión actual de linux.

```
[miguelangel@localhost proyect-one-instance-01]$ hcloud server list
ID          NAME                STATUS    IPV4           IPV6                    PRIVATE NET   LOCATION   AGE
157282133   nivel1-one-instance   running   77.42.89.255   2a01:4f9:c015:b495::/64   -             hel1       5m
[miguelangel@localhost proyect-one-instance-01]$
```

Así como también ver las características de una instancia determinada.

```
[miguelangel@localhost proyect-one-instance-01]$ hcloud server describe nivel1-one-instance
ID:       157282133
Name:     nivel1-one-instance
Status:   running
Created:  2026-07-30 23:48:02 CEST (6 minutes ago)

Server Type:
  ID:            114
  Name:          cx23
  Description:   CX 23
  Category:      cost_optimized
  Cores:         2
  CPU Type:      shared
  Architecture:  x86
  Memory:        4.0 GB
  Disk:          40 GB
  Storage Type:  local

Public Net:
  IPv4:
    ID:       142832205
    IP:       77.42.89.255
    Blocked:  no
    DNS:      static.255.89.42.77.clients.your-server.de

  IPv6:
    ID:       142832206
    IP:       2a01:4f9:c015:b495::/64
    Blocked:  no

  Floating IPs:
    No Floating IPs

Private Net:
  No Private Networks

Volumes:
  No Volumes

Image:
```

Si realizamos una petición Get, mediante curl a la página web desplegada en el servidor web Nginx, se puede ver que esta responde perfectamente.

```
[miguelangel@localhost proyect-one-instance-01]$ curl http://$(terraform output -raw server_ip)
<html>
  <head><title>Nivel 1 - Terraform + Hetzner</title></head>
  <body><h1>Funciona. Nivel 1 desplegado con IaC.</h1></body>
</html>
[miguelangel@localhost proyect-one-instance-01]$
```

Es más, si hacemos ping a esa dirección IP pública, responderá perfectamente.

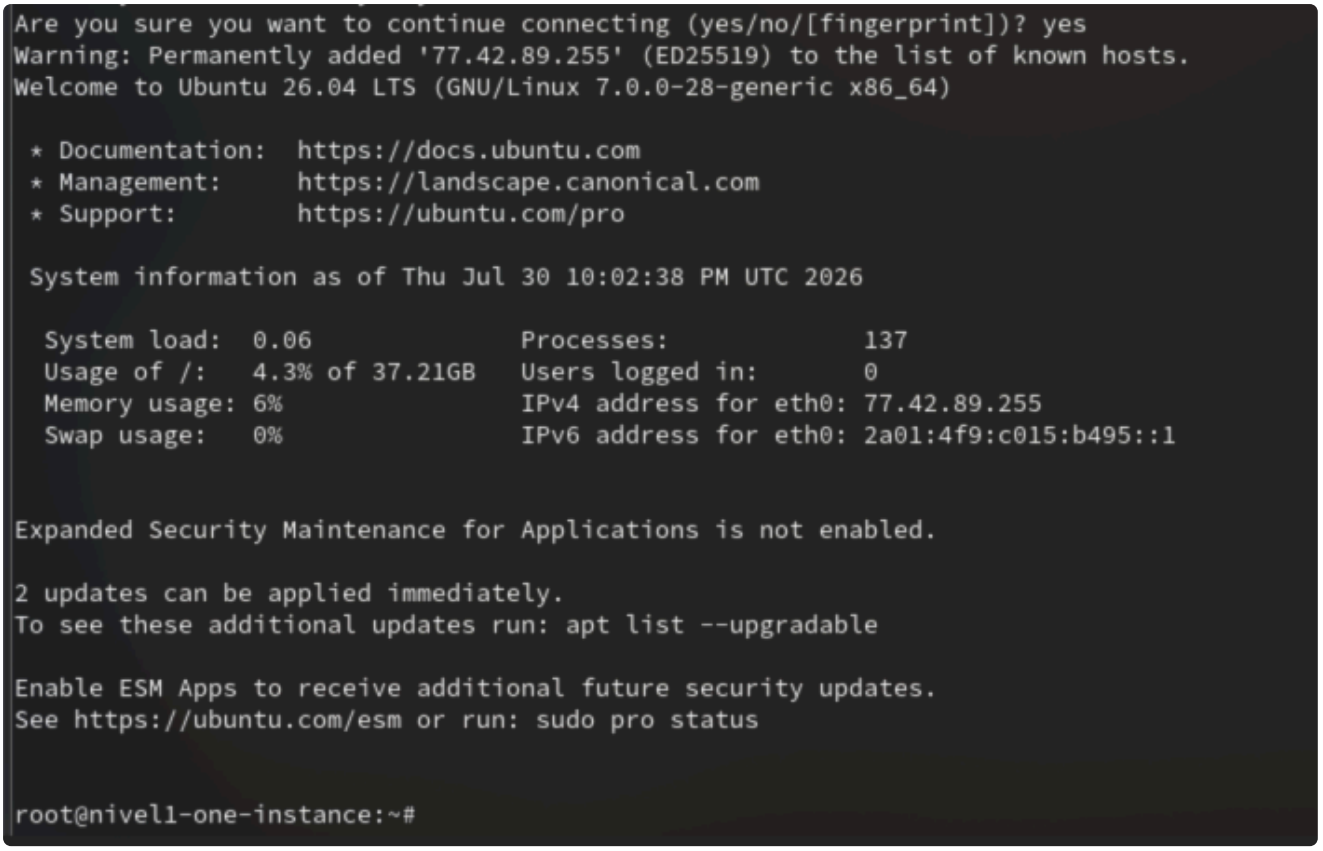
```
Pinging 77.42.89.255 with 32 bytes of data:
Reply from 77.42.89.255: bytes=32 time=69ms TTL=50
Reply from 77.42.89.255: bytes=32 time=65ms TTL=50
Reply from 77.42.89.255: bytes=32 time=67ms TTL=50
Reply from 77.42.89.255: bytes=32 time=66ms TTL=50

Ping statistics for 77.42.89.255:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 65ms, Maximum = 69ms, Average = 66ms

C:\Users\LEGION>
```

Por último, quedaría conectarse a la instancia mediante SSH, esto se hará a través del siguiente comando

```
ssh -i ~/.ssh/hetzner_lab root@$(terraform output -raw server_ip)
```




## Finalización del proyecto

Una vez terminado el proyecto, es necesario borrarlo todo, hay que tener mucho cuidado con este comando. Si es un entorno en producción, la simple ejecución de este comando provocaría la eliminación de toda la infraestructura. Finalmente eliminaremos el proyecto con el comando

```
terraform destroy
```